HPPS Assignment A2

Björn Petterson (zts900), Manar(chs380), Thomas Andersen (kgq325)

April 15, 2026

Introduction

Looking back at the assignment things start to make sense, but while in the process of it, we certainly were frustrated at times. The frustration came from a combination of ambiguity in function descriptions, distinguishing between representation and interpretation as well as general issues getting used to the C syntax. We especially had issues thinking about way to reliable test our implementations.

Steps to compile: Running `$make` without arguments from a terminal will compile our implementations. Running `$make clean` will remove any and all compiled files.

Task 2.1

The functionality of the code `read_unit_be` is correct, it reconstructs a 32-bit integer by reading 4 bytes in big endian order
When implementing `read_unit_be` we find out that it can fail if it reads less than 4 bytes.
In the case of `write_unit_be` we can see that the funcionality of it is correct, it writes a 32 bit integer to the file in big endian order
We here observe that it fails if we write less than 4 bytes and that fwrite can fail if it gets an invalid pointer

Task 2.2

The `read_double_bin` function reads a double value by using "fread". If the file doesn't have enough bytes for a double, or that fread finds any error, then it could possibly lead to a failure.

What i think that my could be wrong is that the function asserts pointers, which could cause the program to terminate if theese assertions fails.
For `write_bouble_bin` we have a function which writes a double value using fwrite. It could fail if we wrote less than the size of a double, or if fwrite finds any error (like an invalid file pointer). It could be missing that it doesn't check for invalid pointers

Task 2.3

As suggested we used the code for `write_uint_ascii` only changing formatting to accommodate that the different in input type (`%f`).

Task 2.4

For this function we re-used the structure from `read_uint_ascii`. To handle the difference between numbers before and after the decimal point we use a variable (`fractional`) to store whether or not a decimal point has been encountered. For each `char` we check if the variable has been set. If it has we sum up the numbers dividing by factors of 10 for each additional digit. We found the description regarding preceding sign ('-') confusing as the information regarding ASCII decimal numbers seemed to indicate non-negative numbers. We did however implement a check for sign to handle negative inputs to the function.

Task 2.5

Here it took some time to find out that the way the assingment wanted us to pass a binary stream to stdin was through the provided piping operation only because this type of advanced bash command was new to me. As a result I fist solved this by generating a binary file with an external script to pass into the function (assisted by GPT). I also had to remember that the bash in itself also is equal to a file that we stream from this might be ovious since we are able to pass in either stdin or a file pointer to the function but non the less also took a while to understand, and hence that the provided piping operation creates a bash file equal to that of a generated binary file passed in as ./program < binary.bin to stdin. With that basic understanding of how to test lets move on to the implementation.

To the assignment: In the implementation we basically loop through the binary doubles provided by the input stream using "read double bin" and in extension "fread".

So lets get into fread. fread and how files actually work, a FILE object contains some meta data to the file that both keep track of state f.ex r or w and where the pointer for r or w currently is in the file. So when we in extention passes a f object to fread, what fread does is advance this pointer in mode r, and return either EOF which is a int constant set by <stdio.h> 0 for success or 1 for error. fread also takes the nr steps to advance the pointer and copys those bits to the space pointed to by the provided pointer here *out.

Lets get back to our "avg doubles" implementation. We use "read double bin" with a pointer to the file stdin and an int pointer we want to read into, here latest val. We continue to advance thsi pointer by calling readdoublebin until it returns EOF or 1 as indicating error. Otherwise if read successful we simply add the read double to a cumulated sum variable and we also increase a counter for how many doubles we have read. With this the average is simply sum/count. We use "write double ascii" to print this to stdout.

Assignment specific questions

1. How much larger are data files in the ASCII integer format, compared to the binary integer format?

We know that a 62-bit int takes up 8 bytes or a 32-bit int takes up 4. We also know that each base-10 ASCII character takes up a byte. That means that a 32-bit int represented as ASCII would take 12 bytes, meaning 3x the space.

2. How much larger are data files in the ASCII floating-point format, compared to the binary floating-point format?

In ASCII format floating point just adds another character (1 byte) on top of however many character are needed for whole numbers and decimals. A single precision float is 32 bits or 4 bytes. To represent this in ASCII you would need differing amount of bytes based on notation. Say you want scientific notation, you could go with '-3.4028234e+38'. For this you would need 14*4 bytes = 56 bytes. This would result in a file size that is increased 14-fold.

3. Would it be possible to add a function read_uint() that accepts both the binary and ASCII integer format; deciding the interpretation based on the actual input? Explain why or why not.

No, it would not be possible to add a function read uint() that accepts both binary and ASCII integer formats and first decides within the function how to interpret the input as ASCII or binary.

First of all, there are some practical difficulties with this question. The idea of adding a function with such functionality is problematic because functions in C typically take arguments with explicitly defined types, not undefined inputs. If we are creative, we could speculate that it might be possible to pass in a void* pointer or simply use either an int or an unsigned char. However, disregarding the format and attempting to extract the underlying bits assumes the user has arbitrarily cast the input into some format before sending it to the function, while that format itself does not inherently convey any meaningful type information.

The second part of the question might hint at: "Is it possible to determine, from the binary representation in C, what type of object something is?" Here, the answer is no. There is no underlying binary pattern that inherently separates integers (int) and ASCII representations of characters. C does not provide any mechanism to infer the type of data from the raw binary representation alone.

4. Are there any floating-point values that can be expressed in the binary format, but not the ASCII format?

No ascii includes the regular signs that we use in daily comunications numbers 1,9 letters, punctuation and symbols, etc. It is generally possible to express any floating point number number using symbols 1 to 9 and a dot. One could even argue that we can express more numbers in ascii than can be stored in the IEEE 754 double format since this has to be rounded at some point. In ascii we could f.ex express an irrational nr like: 2**(1/2) that we could not represent in our binary double format.